

Los tres grandes Anime wiki fan

Guia de estudio del codigo del proyecto Jakarta EE + MySQL + HTML/CSS/JS

Este PDF esta pensado para estudiar el proyecto para un examen: explica que hace cada parte, que archivo llama a cual, como viajan los datos desde la pagina del usuario hasta Java y MySQL, y donde mirar si algo falla.

Nota importante: el proyecto tiene muchas paginas HTML parecidas. Para que sea estudiable, la explicacion es linea a linea por bloques reales y por lineas importantes: cabecera, formularios, scripts, funciones, endpoints, SQL y llamadas entre archivos.

1. Vista general del proyecto

El proyecto es una wiki fan llamada Los tres grandes del anime. La parte visual esta hecha con HTML, CSS y JavaScript. La parte de servidor esta hecha con Java Jakarta Servlet. La base de datos es MySQL y guarda usuarios, contenido publicado y solicitudes enviadas por usuarios.

Capa	Archivos	Funcion
HTML	src/main/webapp/*.html	Define la estructura de cada pantalla: portada, login, registro, dashboard, admin y paginas de anime.
CSS	src/main/webapp/Assets/css/*.css	Define la estetica: fondo oscuro, tarjetas, botones dorados, formularios y responsive.
JavaScript	src/main/webapp/Assets/js/*.js	Controla login, registro, sesiones en localStorage, peticiones fetch, renderizado dinamico y panel admin.
Java	src/main/java/com/ejemplo	Recibe peticiones HTTP, lee/escribe en MySQL y devuelve JSON.
MySQL	usuarios, contenido_wiki, solicitudes_contenido, usuarios_bloqueados	Persiste cuentas, roles, publicaciones y mensajes de usuarios.

2. Mapa de carpetas

Ruta	Que contiene
pom.xml	Configuracion Maven: Java 21, WAR, Jakarta Servlet, MySQL Connector y Gson.
src/main/webapp	Paginas HTML publicas que abre el navegador.
src/main/webapp/Assets/css	Hojas de estilo de login, registro, dashboard, admin y paginas generales.
src/main/webapp/Assets/js	Logica del navegador: formularios, sesion, API, admin y contenido dinamico.
src/main/java/com/ejemplo/controller	Servlets: endpoints HTTP como /auth/login o /api/content.
src/main/java/com/ejemplo/model	Acceso a datos: conexiones, SQL y modelos de usuarios/contenido.
src/main/java/com/ejemplo/filter	Filtro global de cache.
mysql/init/01-bd1.sql	Script inicial de base de datos para Docker/MySQL.

3. Que llama a que

Este es el flujo principal del proyecto. Memorizarlo ayuda mucho para explicar el examen.

Accion	Cadena de llamadas
Iniciar sesion	login.html -> login.js -> fetch /auth/login -> AuthLoginController -> UsuarioLoginModel -> ConexionBD -> tabla usuarios -> JSON -> localStorage -> dashboard/admin.
Crear cuenta	register.html -> register.js -> fetch /auth/register -> AuthRegisterController -> UsuarioRegisterModel -> tabla usuarios -> guarda sesion.
Recuperar contrasena	forgot-password.html -> forgot-password.js -> /auth/recover/check o /auth/recover/reset -> AuthRecoverController -> UsuarioRecoveryModel.
Enviar solicitud	pagina anime -> script.js -> fetch /api/requests POST -> SolicitudesContenidoController -> ContenidoWikiModel.insertarSolicitud -> solicitudes_contenido.
Admin publica contenido	admin.html -> admin.js -> fetch /api/content POST -> ContenidoController -> ContenidoWikiModel.guardarContenido -> contenido_wiki.
Usuario ve contenido	pagina anime -> script.js -> fetch /api/content GET -> ContenidoController -> ContenidoWikiModel.listarContenido -> renderiza bloque Comunidad.
Admin revisa solicitud	admin.html -> admin.js -> fetch /api/requests POST/DELETE -> SolicitudesContenidoController -> ContenidoWikiModel.actualizar/borrarSolicitud.

4. Endpoints Java importantes

Controlador	URL	Metodos HTTP
AuthLoginController.java	/auth/login	doPost
AuthRecoverController.java	urlPatterns = {/auth/recover/check, /auth/recover/reset}	doPost
AuthRegisterController.java	/auth/register	doPost
BloquearUsuarioController.java	/admin/bloquear	doPost
ContenidoController.java	/api/content	doGet, doPost, doDelete
DesbloquearUsuarioController.java	/admin/desbloquear	doPost
ListarBloqueadosController.java	/admin/listarBloqueados	doGet
SolicitudesContenidoController.java	/api/requests	doGet, doPost, doDelete

5. HTML explicado

Los HTML son la cara visible. Normalmente siguen esta estructura: DOCTYPE, html lang='es', head con titulo/CSS, body con cabecera, contenido principal y scripts al final. Los scripts al final son importantes porque así JavaScript encuentra los elementos ya creados.

Tipo de HTML	Archivos	Explicacion
Portada publica	index.html	Presenta la wiki y botones de iniciar sesion/crear cuenta. No requiere usuario.
Autenticacion	login.html, register.html, forgot-password.html	Formularios de usuario. Cargan CSS propio y JS que envia los datos al backend.
Zona privada	dashboard.html	Portada de usuario registrado. Usa auth-helper/script para mostrar usuario, audio, fondo y contenido.
Panel admin	admin.html	Formulario para publicar contenido, lista de solicitudes y gestion de usuarios/roles.
Portadas anime	one-piece.html, naruto.html, bleach.html	Tarjetas de apartados. Usan el mismo sistema visual y script.js.
Paginas internas	one-piece-arcos.html, naruto-equipo-7.html, bleach-bankais.html, etc.	Tienen data-page para saber donde guardar/mostrar contenido y solicitudes.

src/main/webapp/login.html

Linea	Codigo	Que hace
1	<!DOCTYPE html>	indica HTML5.
2	<html lang="es">	abre el documento y declara idioma.
3	<head>	empieza metadatos, titulo y enlaces CSS.
5	<title>Iniciar sesion Los tres grandes del anime</title>	texto de la pestana del navegador.
8	<link rel="stylesheet" href="Assets/css/login.css">	carga una hoja CSS que da estilo a esta pagina.
10	<body class="auth-manga-page">	empieza lo visible; sus clases activan estilos/temas.
11	<div class="login-container">	elemento estructural o clase CSS importante.
12	<section class="login-box">	elemento estructural o clase CSS importante.
13	<p class="login-kicker">Enciclopedia anime</p>	elemento estructural o clase CSS importante.
17	<form id="loginForm" method="POST">	formulario que JavaScript escucha con submit.
18	<div class="input-group">	elemento estructural o clase CSS importante.
20	<input type="text" id="login" placeholder="Correo o nombre de usuario" required>	campo de entrada para usuario/datos.
23	<div class="input-group">	elemento estructural o clase CSS importante.
25	<input type="password" id="password" placeholder="*****" required>	campo de entrada para usuario/datos.

Linea	Codigo	Que hace
28	<code><button type="submit" class="btn-login">Iniciar sesion</button></code>	boton de accion del formulario/interfaz.
30	<code><p id="error" class="error-text"></p></code>	elemento estructural o clase CSS importante.
32	<code><div class="extra-links"></code>	elemento estructural o clase CSS importante.
33	<code>Olvidaste tu contraseña?</code>	elemento estructural o clase CSS importante.
40	<code><script src="Assets/js/auth-helper.js"></script></code>	carga JavaScript; normalmente al final para manipular el DOM.
41	<code><script src="Assets/js/login.js"></script></code>	carga JavaScript; normalmente al final para manipular el DOM.

src/main/webapp/admin.html

Linea	Codigo	Que hace
1	<code><!DOCTYPE html></code>	indica HTML5.
2	<code><html lang="es"></code>	abre el documento y declara idioma.
3	<code><head></code>	empieza metadatos, titulo y enlaces CSS.
6	<code><title>Panel de contenido Los tres grandes del anime</title></code>	texto de la pestana del navegador.
8	<code><link rel="stylesheet" href="Assets/css/dashboard.css"></code>	carga una hoja CSS que da estilo a esta pagina.
10	<code><body class="theme-home detail-page" data-requires-auth="true" data-page="admin" style="--</code>	empieza lo visible; sus clases activan estilos/temas.
11	<code><header class="wiki-header"></code>	empieza metadatos, titulo y enlaces CSS.
12	<code></code>	elemento estructural o clase CSS importante.
13	<code>3</code>	elemento estructural o clase CSS importante.
14	<code></code>	elemento estructural o clase CSS importante.
20	<code><nav class="wiki-nav" aria-label="Navegacion principal"></code>	elemento estructural o clase CSS importante.
27	<code><div class="user-menu" id="userMenu"></code>	elemento estructural o clase CSS importante.
28	<code><button class="user-profile-button" id="userMenuButton" type="button" aria-expanded="false</code>	boton de accion del formulario/interfaz.
29	<code>A</code>	elemento estructural o clase CSS importante.
30	<code></code>	elemento estructural o clase CSS importante.
32	<code>Usuario</code>	elemento estructural o clase CSS importante.
34	<code>+</code>	elemento estructural o clase CSS importante.
37	<code><div class="user-dropdown" id="userDropdown"></div></code>	elemento estructural o clase CSS importante.
41	<code><main class="page-shell"></code>	contenedor principal de la pantalla.
42	<code><section class="detail-hero horizontal-detail"></code>	elemento estructural o clase CSS importante.
43	<code><figure class="detail-cover horizontal-cover"></code>	elemento estructural o clase CSS importante.
47	<code><div class="detail-copy"></code>	elemento estructural o clase CSS importante.
48	<code><p class="eyebrow">Panel abierto con control por roles</p></code>	elemento estructural o clase CSS importante.
51	<code>Volver a la portada principal</code>	elemento estructural o clase CSS importante.
55	<code><section class="stats-strip" id="statsStrip"></section></code>	elemento estructural o clase CSS importante.
57	<code><section class="admin-grid"></code>	elemento estructural o clase CSS importante.
58	<code><article class="detail-panel admin-panel"></code>	elemento estructural o clase CSS importante.
60	<code><p class="eyebrow">Nuevo contenido</p></code>	elemento estructural o clase CSS importante.
64	<code><form class="admin-form" id="adminContentForm"></code>	formulario que JavaScript escucha con submit.
65	<code><input type="hidden" id="adminEntryId"></code>	campo de entrada para usuario/datos.

Línea	Código	Que hace
67	<label class="field">	elemento estructural o clase CSS importante.
69	<select id="adminPageKey" required> </select>	elemento estructural o clase CSS importante.
72	<label class="field wide">	elemento estructural o clase CSS importante.
74	<input id="adminEntryTitle" type="text" placeholder="Titulo del contenido" required>	campo de entrada para usuario/datos.
77	<label class="field wide">	elemento estructural o clase CSS importante.
79	<textarea id="adminEntryBody" rows="7" placeholder="Escribe aqui el contenido que quieres	campo largo de texto.
82	<div class="form-inline-actions">	elemento estructural o clase CSS importante.
83	<button class="button-link primary" type="submit" id="adminSaveButton">Guardar contenido</	boton de accion del formulario/interfaz.

Este archivo tiene mas lineas importantes (50). La idea se repite: clases para CSS, ids para JS y scripts al final.

src/main/webapp/one-piece-arcos.html

Línea	Código	Que hace
1	<!DOCTYPE html>	indica HTML5.
2	<html lang="es">	abre el documento y declara idioma.
3	<head>	empieza metadatos, titulo y enlaces CSS.
6	<title>Arcos de One Piece Los tres grandes del anime</title>	texto de la pestana del navegador.
7	<link rel="stylesheet" href="Assets/css/dashboard.css">	carga una hoja CSS que da estilo a esta pagina.
9	<body class="theme-one-piece detail-page" data-requires-auth="true" data-page="detail" sty	empieza lo visible; sus clases activan estilos/temas.
10	<header class="wiki-header">	empieza metadatos, titulo y enlaces CSS.
11		elemento estructural o clase CSS importante.
12	3	elemento estructural o clase CSS importante.
13		elemento estructural o clase CSS importante.
19	<nav class="wiki-nav" aria-label="Navegacion principal">	elemento estructural o clase CSS importante.
26	<div class="user-menu" id="userMenu">	elemento estructural o clase CSS importante.
27	<button class="user-profile-button" id="userMenuButton" type="button" aria-expanded="false	boton de accion del formulario/interfaz.
28	A	elemento estructural o clase CSS importante.
29		elemento estructural o clase CSS importante.
31	Usuario	elemento estructural o clase CSS importante.
33	 + 	elemento estructural o clase CSS importante.
36	<div class="user-dropdown" id="userDropdown">	elemento estructural o clase CSS importante.
37	<button class="dropdown-item logout-item" id="logoutButton" type="button">Cerrar sesion</b	boton de accion del formulario/interfaz.
42	<main class="page-shell">	contenedor principal de la pantalla.
43	<section class="detail-hero horizontal-detail">	elemento estructural o clase CSS importante.
44	<figure class="detail-cover horizontal-cover">	elemento estructural o clase CSS importante.
48	<div class="detail-copy">	elemento estructural o clase CSS importante.
49	<p class="eyebrow">One Piece</p>	elemento estructural o clase CSS importante.
52	Volver a One Piece	elemento estructural o clase CSS importante.
56	<section class="stats-strip">	elemento estructural o clase CSS importante.
71	<section class="detail-layout">	elemento estructural o clase CSS importante.

Linea	Codigo	Que hace
72	<article class="detail-panel">	elemento estructural o clase CSS importante.
73	<p class="eyebrow">Guia general</p>	elemento estructural o clase CSS importante.
77	<ul class="detail-list">	elemento estructural o clase CSS importante.
84	<div class="content-list">	elemento estructural o clase CSS importante.
85	<section class="community-entry">	elemento estructural o clase CSS importante.
86	<div class="community-entry-head">	elemento estructural o clase CSS importante.
88	<p class="eyebrow">Saga East Blue</p>	elemento estructural o clase CSS importante.
91	6 arcos	elemento estructural o clase CSS importante.
93	<p class="entry-body">La aventura arranca con un tono clasico de piratas y va reuniendo a	elemento estructural o clase CSS importante.
94	<ol class="detail-list" start="1">	elemento estructural o clase CSS importante.
104	<section class="community-entry">	elemento estructural o clase CSS importante.

Este archivo tiene mas lineas importantes (103). La idea se repite: clases para CSS, ids para JS y scripts al final.

src/main/webapp/dashboard.html

Linea	Codigo	Que hace
1	<!DOCTYPE html>	indica HTML5.
2	<html lang="es">	abre el documento y declara idioma.
3	<head>	empieza metadatos, titulo y enlaces CSS.
6	<title>Los tres grandes del anime</title>	texto de la pestana del navegador.
8	<link rel="stylesheet" href="Assets/css/dashboard.css">	carga una hoja CSS que da estilo a esta pagina.
10	<body class="theme-home home-page" data-requires-auth="true" data-page="home">	empieza lo visible; sus clases activan estilos/temas.
11	<header class="wiki-header">	empieza metadatos, titulo y enlaces CSS.
12		elemento estructural o clase CSS importante.
13	3	elemento estructural o clase CSS importante.
14		elemento estructural o clase CSS importante.
20	<nav class="wiki-nav" aria-label="Navegacion principal">	elemento estructural o clase CSS importante.
21	Portada	elemento estructural o clase CSS importante.
27	<div class="user-menu" id="userMenu">	elemento estructural o clase CSS importante.
28	<button class="user-profile-button" id="userMenuButton" type="button" aria-expanded="false	boton de accion del formulario/interfaz.
29	A	elemento estructural o clase CSS importante.
30		elemento estructural o clase CSS importante.
32	Usuario	elemento estructural o clase CSS importante.
34	 + 	elemento estructural o clase CSS importante.
37	<div class="user-dropdown" id="userDropdown">	elemento estructural o clase CSS importante.
38	<button class="dropdown-item logout-item" id="logoutButton" type="button">Cerrar sesion</b	boton de accion del formulario/interfaz.
43	<main class="page-shell">	contenedor principal de la pantalla.
44	<section class="hero-panel hero-panel-stacked">	elemento estructural o clase CSS importante.
45	<div class="hero-panel-head">	elemento estructural o clase CSS importante.
46	<p class="eyebrow">Portada principal</p>	elemento estructural o clase CSS importante.
50	<figure class="hero-panel-cover">	elemento estructural o clase CSS importante.

Línea	Código	Que hace
54	<div class="hero-panel-body">	elemento estructural o clase CSS importante.
55	<div class="hero-panel-copy">	elemento estructural o clase CSS importante.
56	<p class="hero-text">	elemento estructural o clase CSS importante.
63	<div class="hero-feature-strip">	elemento estructural o clase CSS importante.
79	<section class="section-block">	elemento estructural o clase CSS importante.
80	<div class="section-heading">	elemento estructural o clase CSS importante.
81	<p class="eyebrow">Series destacadas</p>	elemento estructural o clase CSS importante.
85	<div class="anime-grid">	elemento estructural o clase CSS importante.
86		elemento estructural o clase CSS importante.
87	<figure class="card-media">	elemento estructural o clase CSS importante.
88		elemento estructural o clase CSS importante.
93	Abrir portada	elemento estructural o clase CSS importante.

Este archivo tiene mas líneas importantes (49). La idea se repite: clases para CSS, ids para JS y scripts al final.

6. CSS explicado

El CSS crea la estética real del proyecto. La identidad visual se basa en fondo azul/negro, tarjetas con bordes suaves, botones dorados, radios grandes y tipografía de título tipo serif.

Archivo CSS	Uso
dashboard.css	Estilo principal de la wiki: header, hero, tarjetas, layout responsive, comunidad, audio y botones.
admin.css	Panel de administrador: formularios, columnas, listas, roles, solicitudes y entradas publicadas.
login.css	Pantalla de login: tarjeta central, fondo oscuro, formulario y enlaces.
register.css	Pantalla crear cuenta, campos y botones.
forgot-password.css	Recuperación de cuenta con tarjeta centrada y estados de cuenta encontrada.
estilos.css	CSS adicional/legacy para partes generales del sitio.

src/main/webapp/Assets/css/dashboard.css

Línea	Selector	Que controla
1	*	bloque de estilos.
5	:root	variables globales: colores, sombras, radios y medidas reutilizables.
19	body.theme-home	fondo, fuente base y color general.
23	body.theme-one-piece	fondo, fuente base y color general.
27	body.theme-naruto	fondo, fuente base y color general.
31	body.theme-bleach	fondo, fuente base y color general.
35	html	bloque de estilos.
39	body	fondo, fuente base y color general.
48	body::before	fondo, fuente base y color general.
62	body::after	fondo, fuente base y color general.
72	a	bloque de estilos.

Línea	Selector	Que controla
77	button, input, textarea, select	botones, normalmente dorados o secundarios.
84	.wiki-header	barra superior y navegacion.
98	.brand	bloque de estilos.
104	.brand-mark	bloque de estilos.
121	.brand-copy strong, .brand-copy small	bloque de estilos.
126	.brand-copy strong	bloque de estilos.
131	.brand-copy small	bloque de estilos.
136	.wiki-nav	barra superior y navegacion.
143	.wiki-nav a	barra superior y navegacion.
151	.wiki-nav a:hover, .wiki-nav a.is-active	barra superior y navegacion.
158	.user-menu	bloque de estilos.
162	.user-profile-button	botones, normalmente dorados o secundarios.
174	.user-avatar	bloque de estilos.
185	.user-details	bloque de estilos.
190	.user-details small	bloque de estilos.
195	.user-details span	bloque de estilos.
203	.user-arrow	bloque de estilos.
208	.user-dropdown	bloque de estilos.
224	.user-dropdown.show	bloque de estilos.
230	.dropdown-item	bloque de estilos.
241	.dropdown-item:hover	bloque de estilos.
245	.dropdown-link	bloque de estilos.
252	.dropdown-link:hover	bloque de estilos.
256	.page-shell	bloque de estilos.
262	.hero-panel, .series-hero, .detail-hero, .section-block, .detail-panel	tarjetas/paneles oscuros con borde.
273	.hero-panel	tarjetas/paneles oscuros con borde.
281	.hero-panel-stacked	tarjetas/paneles oscuros con borde.
286	.hero-panel-head	tarjetas/paneles oscuros con borde.
291	.hero-panel-cover	tarjetas/paneles oscuros con borde.
300	.hero-panel-cover img	tarjetas/paneles oscuros con borde.
308	.hero-panel-body	fondo, fuente base y color general.
315	.hero-panel-copy	tarjetas/paneles oscuros con borde.
320	.series-hero, .detail-hero	bloque de estilos.
328	.series-cover, .detail-cover, .card-media	tarjetas/paneles oscuros con borde.

Hay 256 selectores en total. En el archivo real se repiten patrones: layout, tarjetas, botones, estados y responsive.

src/main/webapp/Assets/css/admin.css

Línea	Selector	Que controla
1	*	bloque de estilos.
6	:root	variables globales: colores, sombras, radios y medidas reutilizables.
21	body	fondo, fuente base y color general.
28	button, input, select	botones, normalmente dorados o secundarios.
34	button	botones, normalmente dorados o secundarios.
38	.admin-layout	bloque de estilos.
44	.sidebar	bloque de estilos.
57	.brand	bloque de estilos.
63	.brand-icon	bloque de estilos.
75	.brand strong, .brand small	bloque de estilos.
81	.brand strong	bloque de estilos.
86	.brand small	bloque de estilos.
92	.nav	barra superior y navegacion.
98	.nav button, .button	barra superior y navegacion.
110	.nav button	barra superior y navegacion.
114	.nav button:hover, .nav button.active, .button.primary	barra superior y navegacion.
122	.button:hover	botones, normalmente dorados o secundarios.
126	.button.danger	botones, normalmente dorados o secundarios.
132	.button.danger:hover	botones, normalmente dorados o secundarios.
138	.button.success	botones, normalmente dorados o secundarios.
144	.sidebar-actions	bloque de estilos.
149	.main	bloque de estilos.
154	.topbar	bloque de estilos.
162	.eyebrow	bloque de estilos.
169	h1	bloque de estilos.
175	h2	bloque de estilos.
179	.muted	bloque de estilos.
184	.stats	bloque de estilos.
191	.stat, .panel	tarjetas/paneles oscuros con borde.
199	.stat	bloque de estilos.
203	.stat span	bloque de estilos.
208	.stat strong	bloque de estilos.
215	.toolbar	bloque de estilos.
224	.filters	bloque de estilos.
230	.field	bloque de estilos.
238	input, select	campos de formulario.

Línea	Selector	Que controla
248	.view	bloque de estilos.
252	.view.active	bloque de estilos.
256	.table-wrap	bloque de estilos.
260	table	bloque de estilos.
266	th, td	bloque de estilos.
275	th	bloque de estilos.
282	tr:last-child td	bloque de estilos.
286	.badge	bloque de estilos.
299	.badge.success	bloque de estilos.

Hay 64 selectores en total. En el archivo real se repiten patrones: layout, tarjetas, botones, estados y responsive.

src/main/webapp/Assets/css/login.css

Línea	Selector	Que controla
1	*	bloque de estilos.
5	:root	variables globales: colores, sombras, radios y medidas reutilizables.
19	body	fondo, fuente base y color general.
36	body.auth-manga-page	fondo, fuente base y color general.
43	.login-container	bloque de estilos.
47	.site-footer	bloque de estilos.
58	.site-footer-inner	bloque de estilos.
64	.site-footer-copy	bloque de estilos.
70	.site-footer-note	bloque de estilos.
76	.login-box	bloque de estilos.
85	.recover-box	bloque de estilos.
89	.login-kicker	bloque de estilos.
98	.login-box h1	bloque de estilos.
105	.login-box > p	bloque de estilos.
111	.input-group	campos de formulario.
115	.input-group label	campos de formulario.
123	.input-group input	campos de formulario.
133	.input-group input:focus	campos de formulario.
138	.btn-login	botones, normalmente dorados o secundarios.
151	.btn-login:hover	botones, normalmente dorados o secundarios.
156	.error-text	bloque de estilos.
162	.recover-message.error, #error	bloque de estilos.
167	.recover-message.success	bloque de estilos.
171	.extra-links	bloque de estilos.
179	.extra-links a	bloque de estilos.
184	.extra-links a:hover	bloque de estilos.
188	.hidden	bloque de estilos.

Línea	Selector	Que controla
192	.account-found	bloque de estilos.
202	.account-found strong	bloque de estilos.
210	body	fondo, fuente base y color general.
216	body.auth-manga-page	fondo, fuente base y color general.
223	.login-box	bloque de estilos.
228	.extra-links	bloque de estilos.
232	.site-footer	bloque de estilos.

src/main/webapp/Assets/css/forgot-password.css

Línea	Selector	Que controla
1	.recover-container	bloque de estilos.
3	.recover-box	bloque de estilos.
7	.recover-box h1, .recover-box > p	bloque de estilos.
12	.recover-box > p	bloque de estilos.
16	.hidden	bloque de estilos.
20	.account-found	bloque de estilos.
31	.account-found strong	bloque de estilos.
37	.recover-message	bloque de estilos.
46	.recover-message.error	bloque de estilos.
50	.recover-message.success	bloque de estilos.
54	.recover-links	bloque de estilos.
60	body	fondo, fuente base y color general.
65	.recover-links	bloque de estilos.

7. JavaScript explicado

JavaScript es el puente entre lo que ves en pantalla y el servidor. Lee formularios, guarda la sesión en localStorage, llama al backend con fetch y después pinta la respuesta en el HTML.

src/main/webapp/Assets/js/auth-helper.js

Línea	Función / llamada	Explicación
1	const LOCAL_USERS_STORAGE_KEY = "anime-local-users-v1";	función auxiliar o inicializador de interfaz.
2	const AUTH_STORAGE_KEY = "anime-wiki-authenticated";	función auxiliar o inicializador de interfaz.
3	const USERNAME_STORAGE_KEY = "anime-wiki-username";	función auxiliar o inicializador de interfaz.
4	const USER_EMAIL_STORAGE_KEY = "anime-wiki-user-email";	función auxiliar o inicializador de interfaz.
5	const USER_ROLE_STORAGE_KEY = "anime-wiki-user-role";	función auxiliar o inicializador de interfaz.
6	const SITE_FAVICON_PATH = "Assets/img/logo-circle.png?v=1";	función auxiliar o inicializador de interfaz.
7	function normalize(value) {	función auxiliar o inicializador de interfaz.
11	function setSiteFavicon() {	función auxiliar o inicializador de interfaz.
16	function setFaviconLink(relValue) {	función auxiliar o inicializador de interfaz.
29	function readUsers() {	función auxiliar o inicializador de interfaz.
32	const raw = localStorage.getItem(LOCAL_USERS_STORAGE_KEY);	guarda o lee datos persistentes del navegador.

Línea	Funcion / llamada	Explicacion
33	const users = raw ? JSON.parse(raw) : [];	funcion auxiliar o inicializador de interfaz.
39	function writeUsers(users) {	funcion auxiliar o inicializador de interfaz.
43	function ensureSeedUsers() {	funcion auxiliar o inicializador de interfaz.
45	const users = readUsers();	funcion auxiliar o inicializador de interfaz.
46	const seeds = [	funcion auxiliar o inicializador de interfaz.
68	const exists = users.some((user) => normalize(user.email) === normalize(seed.email));	funcion auxiliar o inicializador de interfaz.
79	function getCurrentUser() {	funcion auxiliar o inicializador de interfaz.
81	const email = localStorage.getItem(USER_EMAIL_STORAGE_KEY);	guarda o lee datos persistentes del navegador.
85	function findByLogin(login) {	parte del flujo de inicio de sesion.
87	const key = normalize(login);	parte del flujo de inicio de sesion.
92	function findByEmail(email) {	funcion auxiliar o inicializador de interfaz.
94	const key = normalize(email);	funcion auxiliar o inicializador de interfaz.
97	function saveSession(user) {	funcion auxiliar o inicializador de interfaz.
104	function clearSession() {	funcion auxiliar o inicializador de interfaz.
111	function upsertUser(data) {	funcion auxiliar o inicializador de interfaz.
113	const users = readUsers();	funcion auxiliar o inicializador de interfaz.
114	const index = users.findIndex((user) => normalize(user.email) === normalize(data.email));	funcion auxiliar o inicializador de interfaz.
115	const nextUser = {	funcion auxiliar o inicializador de interfaz.
133	function registerLocalUser(data) {	parte del flujo de registro.
135	const username = String(data.username "").trim();	funcion auxiliar o inicializador de interfaz.
136	const email = String(data.email "").trim().toLowerCase();	funcion auxiliar o inicializador de interfaz.
137	const password = String(data.password "");	funcion auxiliar o inicializador de interfaz.
142	const users = readUsers();	funcion auxiliar o inicializador de interfaz.
144	const duplicate = users.find((user) =>	funcion auxiliar o inicializador de interfaz.
152	const user = upsertUser({ username, email, password, role: "user" });	funcion auxiliar o inicializador de interfaz.
156	function verifyLocalUser(login, password) {	parte del flujo de inicio de sesion.
158	const user = findByLogin(login);	parte del flujo de inicio de sesion.
166	function updatePassword(email, password) {	funcion auxiliar o inicializador de interfaz.
168	const user = findByEmail(email);	funcion auxiliar o inicializador de interfaz.
173	const updatedUser = upsertUser({ ...user, password });	funcion auxiliar o inicializador de interfaz.
177	function listUsers() {	funcion auxiliar o inicializador de interfaz.
184	function updateUserRole(userId, role) {	funcion auxiliar o inicializador de interfaz.
186	const safeRole = String(role "user").toLowerCase();	funcion auxiliar o inicializador de interfaz.
187	const users = readUsers();	funcion auxiliar o inicializador de interfaz.
188	const index = users.findIndex((user) => String(user.id) === String(userId));	funcion auxiliar o inicializador de interfaz.
196	const currentUser = getCurrentUser();	funcion auxiliar o inicializador de interfaz.
204	function deleteUser(userId) {	funcion auxiliar o inicializador de interfaz.
206	const users = readUsers();	funcion auxiliar o inicializador de interfaz.
207	const target = users.find((user) => String(user.id) === String(userId));	funcion auxiliar o inicializador de interfaz.
208	const currentUser = getCurrentUser();	funcion auxiliar o inicializador de interfaz.

Linea	Funcion / llamada	Explicacion
217	<code>const adminCount = users.filter((user) => String(user.role).toLowerCase() === "admin").length;</code>	funcion auxiliar o inicializador de interfaz.

src/main/webapp/Assets/js/login.js

Linea	Funcion / llamada	Explicacion
1	<code>document.addEventListener("DOMContentLoaded", () => {</code>	se ejecuta cuando el HTML ya esta cargado.
2	<code>const form = document.getElementById("loginForm");</code>	parte del flujo de inicio de sesion.
4	<code>const error = document.getElementById("error");</code>	funcion auxiliar o inicializador de interfaz.
8	<code>const login = document.getElementById("login").value.trim();</code>	parte del flujo de inicio de sesion.
10	<code>const password = document.getElementById("password").value.trim();</code>	funcion auxiliar o inicializador de interfaz.
16	<code>const remoteResult = await tryRemoteLogin(login, password);</code>	parte del flujo de inicio de sesion.
20	<code>const remoteUser = {</code>	funcion auxiliar o inicializador de interfaz.
35	<code>const localResult = window.AuthHelper</code>	funcion auxiliar o inicializador de interfaz.
49	<code>function getApiBase() {</code>	funcion auxiliar o inicializador de interfaz.
55	<code>async function tryRemoteLogin(login, password) {</code>	parte del flujo de inicio de sesion.
58	<code>const res = await fetch(`\${getApiBase()}/auth/login`, {</code>	llamada HTTP al backend Java.
65	<code>const data = await safeJson(res);</code>	intenta convertir la respuesta del servidor a JSON sin romper la pagina.
88	<code>async function safeJson(response) {</code>	intenta convertir la respuesta del servidor a JSON sin romper la pagina.
96	<code>function redirectAfterLogin() {</code>	parte del flujo de inicio de sesion.
100	<code>function injectSiteFooter() {</code>	funcion auxiliar o inicializador de interfaz.
105	<code>const year = new Date().getFullYear();</code>	funcion auxiliar o inicializador de interfaz.
107	<code>const footer = document.createElement("footer");</code>	funcion auxiliar o inicializador de interfaz.

src/main/webapp/Assets/js/register.js

Linea	Funcion / llamada	Explicacion
1	<code>document.addEventListener("DOMContentLoaded", () => {</code>	se ejecuta cuando el HTML ya esta cargado.
2	<code>const error = document.getElementById("error");</code>	funcion auxiliar o inicializador de interfaz.
4	<code>const form = document.getElementById("registroForm");</code>	funcion auxiliar o inicializador de interfaz.
8	<code>const username = document.getElementById("nombre").value.trim();</code>	funcion auxiliar o inicializador de interfaz.
10	<code>const email = document.getElementById("email").value.trim().toLowerCase();</code>	funcion auxiliar o inicializador de interfaz.
11	<code>const password = document.getElementById("password").value.trim();</code>	funcion auxiliar o inicializador de interfaz.
12	<code>const confirm = document.getElementById("confirm").value.trim();</code>	funcion auxiliar o inicializador de interfaz.
23	<code>const remoteResult = await tryRemoteRegister({ username, email, password });</code>	parte del flujo de registro.
30	<code>const localResult = window.AuthHelper</code>	funcion auxiliar o inicializador de interfaz.
45	<code>function getApiBase() {</code>	funcion auxiliar o inicializador de interfaz.
51	<code>async function tryRemoteRegister(payload) {</code>	parte del flujo de registro.
54	<code>const response = await fetch(`\${getApiBase()}/auth/register`, {</code>	llamada HTTP al backend Java.
61	<code>const data = await safeJson(response);</code>	intenta convertir la respuesta del servidor a JSON sin romper la pagina.
81	<code>async function safeJson(response) {</code>	intenta convertir la respuesta del servidor a JSON sin romper la pagina.

Línea	Funcion / llamada	Explicacion
89	function finishRegistration(user) {	funcion auxiliar o inicializador de interfaz.
92	const savedUser = window.AuthHelper.upsertUser({ ...user, role: "user" });	funcion auxiliar o inicializador de interfaz.
98	function injectSiteFooter() {	funcion auxiliar o inicializador de interfaz.
103	const year = new Date().getFullYear();	funcion auxiliar o inicializador de interfaz.
105	const footer = document.createElement("footer");	funcion auxiliar o inicializador de interfaz.

src/main/webapp/Assets/js/forgot-password.js

Línea	Funcion / llamada	Explicacion
1	document.addEventListener("DOMContentLoaded", () => {	se ejecuta cuando el HTML ya esta cargado.
2	const apiBase = getApiBase();	funcion auxiliar o inicializador de interfaz.
4	const checkForm = document.getElementById("checkAccountForm");	funcion auxiliar o inicializador de interfaz.
5	const resetForm = document.getElementById("resetPasswordForm");	parte de recuperacion/cambio de contrasena.
6	const emailInput = document.getElementById("recoverEmail");	parte de recuperacion/cambio de contrasena.
7	const newPasswordInput = document.getElementById("newPassword");	funcion auxiliar o inicializador de interfaz.
8	const confirmPasswordInput = document.getElementById("confirmPassword");	funcion auxiliar o inicializador de interfaz.
9	const accountFound = document.getElementById("accountFound");	funcion auxiliar o inicializador de interfaz.
10	const message = document.getElementById("recoverMessage");	parte de recuperacion/cambio de contrasena.
23	const remoteResult = await tryRemoteCheck(apiBase, selectedEmail);	funcion auxiliar o inicializador de interfaz.
30	const localUser = window.AuthHelper ? window.AuthHelper.findByEmail(selectedEmail) : null;	funcion auxiliar o inicializador de interfaz.
44	const password = newPasswordInput.value.trim();	funcion auxiliar o inicializador de interfaz.
46	const confirm = confirmPasswordInput.value.trim();	funcion auxiliar o inicializador de interfaz.
58	const remoteResult = await tryRemoteReset(apiBase, selectedEmail, password);	parte de recuperacion/cambio de contrasena.
62	const localReset = window.AuthHelper.updatePassword(selectedEmail, password);	parte de recuperacion/cambio de contrasena.
77	function showAccount(name, email) {	funcion auxiliar o inicializador de interfaz.
89	function setMessage(text, type) {	funcion auxiliar o inicializador de interfaz.
94	function escapeHtml(value) {	funcion auxiliar o inicializador de interfaz.
104	function getApiBase() {	funcion auxiliar o inicializador de interfaz.
110	async function tryRemoteCheck(apiBase, email) {	funcion auxiliar o inicializador de interfaz.
113	const res = await fetch(`\${apiBase}/auth/recover/check`, {	llamada HTTP al backend Java.
118	const data = await safeJson(res);	intenta convertir la respuesta del servidor a JSON sin romper la pagina.
130	async function tryRemoteReset(apiBase, email, password) {	parte de recuperacion/cambio de contrasena.
133	const res = await fetch(`\${apiBase}/auth/recover/reset`, {	llamada HTTP al backend Java.
138	const data = await safeJson(res);	intenta convertir la respuesta del servidor a JSON sin romper la pagina.
150	async function safeJson(response) {	intenta convertir la respuesta del servidor a JSON sin romper la pagina.
158	function injectSiteFooter() {	funcion auxiliar o inicializador de interfaz.
163	const year = new Date().getFullYear();	funcion auxiliar o inicializador de interfaz.
165	const footer = document.createElement("footer");	funcion auxiliar o inicializador de interfaz.

src/main/webapp/Assets/js/script.js

Línea	Funcion / llamada	Explicacion
1	const AUTH_STORAGE_KEY = "anime-wiki-authenticated";	funcion auxiliar o inicializador de interfaz.
1	const USERNAME_STORAGE_KEY = "anime-wiki-username";	funcion auxiliar o inicializador de interfaz.
2	const USER_EMAIL_STORAGE_KEY = "anime-wiki-user-email";	funcion auxiliar o inicializador de interfaz.
3	const USER_ROLE_STORAGE_KEY = "anime-wiki-user-role";	funcion auxiliar o inicializador de interfaz.
4	const CONTENT_STORAGE_KEY = "anime-page-content-v1";	gestiona entradas de contenido de la comunidad.
5	const CONTENT_REQUESTS_STORAGE_KEY = "anime-page-requests-v1";	gestiona entradas de contenido de la comunidad.
6	const SITE_FAVICON_PATH = "Assets/img/logo-circle.png?v=1";	funcion auxiliar o inicializador de interfaz.
7	const PAGE_AUDIO_CONFIGS = [	funcion auxiliar o inicializador de interfaz.
33	const SERIES_ASIDE_VIDEO_CONFIGS = [	funcion auxiliar o inicializador de interfaz.
92	function redirectToLogin() {	parte del flujo de inicio de sesion.
96	function isAuthenticated() {	funcion auxiliar o inicializador de interfaz.
100	function getCurrentRole() {	funcion auxiliar o inicializador de interfaz.
104	function getCurrentUserName() {	funcion auxiliar o inicializador de interfaz.
108	function getCurrentUserEmail() {	funcion auxiliar o inicializador de interfaz.
112	function getApiBase() {	funcion auxiliar o inicializador de interfaz.
118	async function safeJson(response) {	intenta convertir la respuesta del servidor a JSON sin romper la pagina.
126	async function syncCommunityStateFromServer() {	funcion auxiliar o inicializador de interfaz.
130	fetch(`\${getApiBase()}/api/content`,	llamada HTTP al backend Java.
131	fetch(`\${getApiBase()}/api/requests`)	llamada HTTP al backend Java.
135	const content = await safeJson(contentResponse);	intenta convertir la respuesta del servidor a JSON sin romper la pagina.
142	const requests = await safeJson(requestsResponse);	intenta convertir la respuesta del servidor a JSON sin romper la pagina.
151	async function persistContentEntry(entry) {	gestiona entradas de contenido de la comunidad.
154	const response = await fetch(`\${getApiBase()}/api/content`, {	llamada HTTP al backend Java.
159	const saved = await safeJson(response);	intenta convertir la respuesta del servidor a JSON sin romper la pagina.
168	async function removeContentEntryRemote(entryId) {	gestiona entradas de contenido de la comunidad.
171	await fetch(`\${getApiBase()}/api/content?id=\${encodeURIComponent(entryId)}`, {	llamada HTTP al backend Java.
178	async function persistContentRequest(request) {	gestiona entradas de contenido de la comunidad.
181	const response = await fetch(`\${getApiBase()}/api/requests`, {	llamada HTTP al backend Java.
186	const saved = await safeJson(response);	intenta convertir la respuesta del servidor a JSON sin romper la pagina.
195	function canManageContent() {	gestiona entradas de contenido de la comunidad.
199	function canDeleteContent() {	gestiona entradas de contenido de la comunidad.
203	function canSendContentRequests() {	gestiona entradas de contenido de la comunidad.
207	function ensurePageAccess() {	funcion auxiliar o inicializador de interfaz.
223	function readContentEntries() {	gestiona entradas de contenido de la comunidad.
226	const raw = JSON.parse(localStorage.getItem(CONTENT_STORAGE_KEY) "[]");	guarda o lee datos persistentes del navegador.
232	function saveContentEntries(entries) {	gestiona entradas de contenido de la comunidad.

Linea	Funcion / llamada	Explicacion
236	function readContentRequests() {	gestiona entradas de contenido de la comunidad.
239	const raw = JSON.parse(localStorage.getItem(CONTENT_REQUESTS_STORAGE_KEY) "[]");	guarda o lee datos persistentes del navegador.
245	function saveContentRequests(requests) {	gestiona entradas de contenido de la comunidad.
249	function createContentRequest(request) {	gestiona entradas de contenido de la comunidad.
251	const requests = readContentRequests();	gestiona entradas de contenido de la comunidad.
256	function upsertContentEntry(entry) {	gestiona entradas de contenido de la comunidad.
258	const entries = readContentEntries();	gestiona entradas de contenido de la comunidad.
259	const index = entries.findIndex((item) => String(item.id) === String(entry.id));	gestiona entradas de contenido de la comunidad.
269	function deleteContentEntry(entryId) {	gestiona entradas de contenido de la comunidad.
273	function replaceContentEntry(previousId, entry) {	gestiona entradas de contenido de la comunidad.
275	const entries = readContentEntries().filter((item) => String(item.id) !== String(previousId));	gestiona entradas de contenido de la comunidad.
276	const index = entries.findIndex((item) => String(item.id) === String(entry.id));	gestiona entradas de contenido de la comunidad.
286	function escapeHTML(value) {	funcion auxiliar o inicializador de interfaz.
295	function formatDate(value) {	funcion auxiliar o inicializador de interfaz.
297	const date = new Date(value);	funcion auxiliar o inicializador de interfaz.
307	function setSiteFavicon() {	funcion auxiliar o inicializador de interfaz.
312	function setFaviconLink(relValue) {	funcion auxiliar o inicializador de interfaz.
325	function injectSiteFooter() {	funcion auxiliar o inicializador de interfaz.

Este archivo tiene muchas funciones (150). Las principales ya estan listadas; el resto son auxiliares de renderizado, fechas, escape HTML y eventos.

src/main/webapp/Assets/js/admin.js

Linea	Funcion / llamada	Explicacion
1	const AUTH_STORAGE_KEY = "anime-wiki-authenticated";	funcion auxiliar o inicializador de interfaz.
1	const USERNAME_STORAGE_KEY = "anime-wiki-username";	funcion auxiliar o inicializador de interfaz.
2	const USER_ROLE_STORAGE_KEY = "anime-wiki-user-role";	funcion auxiliar o inicializador de interfaz.
3	const USER_EMAIL_STORAGE_KEY = "anime-wiki-user-email";	funcion auxiliar o inicializador de interfaz.
4	const CONTENT_STORAGE_KEY = "anime-page-content-v1";	gestiona entradas de contenido de la comunidad.
5	const CONTENT_REQUESTS_STORAGE_KEY = "anime-page-requests-v1";	gestiona entradas de contenido de la comunidad.
6	const PAGE_OPTIONS = [	funcion auxiliar o inicializador de interfaz.
27	function isAuthenticated() {	funcion auxiliar o inicializador de interfaz.
31	function getCurrentRole() {	funcion auxiliar o inicializador de interfaz.
35	function getCurrentUserName() {	funcion auxiliar o inicializador de interfaz.
39	function getCurrentUserEmail() {	funcion auxiliar o inicializador de interfaz.
43	function getApiBase() {	funcion auxiliar o inicializador de interfaz.
49	async function safeJson(response) {	intenta convertir la respuesta del servidor a JSON sin romper la pagina.
57	async function syncAdminStateFromServer() {	funcion auxiliar o inicializador de interfaz.
61	fetch(`\${getApiBase()}/api/content`,	llamada HTTP al backend Java.
62	fetch(`\${getApiBase()}/api/requests`)	llamada HTTP al backend Java.

Línea	Funcion / llamada	Explicacion
66	const content = await safeJson(contentResponse);	intenta convertir la respuesta del servidor a JSON sin romper la pagina.
73	const requests = await safeJson(requestsResponse);	intenta convertir la respuesta del servidor a JSON sin romper la pagina.
82	async function persistEntry(entry) {	gestiona entradas de contenido de la comunidad.
85	const response = await fetch(`\${getApiBase()}/api/content`, {	llamada HTTP al backend Java.
90	const saved = await safeJson(response);	intenta convertir la respuesta del servidor a JSON sin romper la pagina.
99	async function removeEntryRemote(entryId) {	gestiona entradas de contenido de la comunidad.
102	await fetch(`\${getApiBase()}/api/content?id=\${encodeURIComponent(entryId)}`, {	llamada HTTP al backend Java.
109	async function persistRequestStatus(requestId, status) {	gestiona solicitudes enviadas por usuarios.
112	await fetch(`\${getApiBase()}/api/requests`, {	llamada HTTP al backend Java.
121	async function removeRequestRemote(requestId) {	gestiona solicitudes enviadas por usuarios.
124	await fetch(`\${getApiBase()}/api/requests?id=\${encodeURIComponent(requestId)}`, {	llamada HTTP al backend Java.
131	function canDeleteContent() {	gestiona entradas de contenido de la comunidad.
135	function roleLabel(role) {	funcion auxiliar o inicializador de interfaz.
137	const labels = {	funcion auxiliar o inicializador de interfaz.
144	function escapeHTML(value) {	funcion auxiliar o inicializador de interfaz.
153	function formatDateTime(value) {	funcion auxiliar o inicializador de interfaz.
155	const date = new Date(value);	funcion auxiliar o inicializador de interfaz.
165	function injectSiteFooter() {	funcion auxiliar o inicializador de interfaz.
170	const year = new Date().getFullYear();	funcion auxiliar o inicializador de interfaz.
172	const footer = document.createElement("footer");	funcion auxiliar o inicializador de interfaz.
183	function readEntries() {	funcion auxiliar o inicializador de interfaz.
186	const raw = JSON.parse(localStorage.getItem(CONTENT_STORAGE_KEY) "[]");	guarda o lee datos persistentes del navegador.
192	function saveEntries(entries) {	funcion auxiliar o inicializador de interfaz.
196	function readRequests() {	gestiona solicitudes enviadas por usuarios.
199	const raw = JSON.parse(localStorage.getItem(CONTENT_REQUESTS_STORAGE_KEY) "[]");	guarda o lee datos persistentes del navegador.
205	function saveRequests(requests) {	gestiona solicitudes enviadas por usuarios.
209	function updateRequestStatus(requestId, status) {	gestiona solicitudes enviadas por usuarios.
211	const requests = readRequests();	gestiona solicitudes enviadas por usuarios.
212	const index = requests.findIndex((request) => String(request.id) === String(requestId));	gestiona solicitudes enviadas por usuarios.
221	function deleteRequest(requestId) {	gestiona solicitudes enviadas por usuarios.
225	function upsertEntry(entry) {	gestiona entradas de contenido de la comunidad.
227	const entries = readEntries();	funcion auxiliar o inicializador de interfaz.
228	const index = entries.findIndex((item) => String(item.id) === String(entry.id));	gestiona entradas de contenido de la comunidad.
236	function deleteEntry(entryId) {	gestiona entradas de contenido de la comunidad.
240	function replaceEntry(previousId, entry) {	gestiona entradas de contenido de la comunidad.
242	const entries = readEntries().filter((item) => String(item.id) !== String(previousId));	funcion auxiliar o inicializador de interfaz.

Linea	Funcion / llamada	Explicacion
243	const index = entries.findIndex((item) => String(item.id) === String(entry.id));	gestiona entradas de contenido de la comunidad.
251	function ensureAccess() {	funcion auxiliar o inicializador de interfaz.

Este archivo tiene muchas funciones (118). Las principales ya estan listadas; el resto son auxiliares de renderizado, fechas, escape HTML y eventos.

8. Java explicado

Java recibe las peticiones que envia fetch. Cada controlador tiene una URL con @WebServlet. Dentro de doGet/doPost/doDelete lee parametros o JSON, llama a un modelo y devuelve JSON con Gson.

src/main/java/com/ejemplo/controller/AuthLoginController.java

Linea	Codigo importante	Que hace
1	package com.ejemplo.controller;	paquete donde vive la clase.
1	import com.ejemplo.model.UsuarioLoginModel;	clase externa que se usa en este archivo.
3	import jakarta.json.Json;	clase externa que se usa en este archivo.
4	import jakarta.json.JsonObject;	clase externa que se usa en este archivo.
5	import jakarta.servlet.annotation.WebServlet;	clase externa que se usa en este archivo.
6	import jakarta.servlet.http.HttpServlet;	clase externa que se usa en este archivo.
7	import jakarta.servlet.http.HttpServletRequest;	clase externa que se usa en este archivo.
8	import jakarta.servlet.http.HttpServletResponse;	clase externa que se usa en este archivo.
9	import jakarta.servlet.http.HttpSession;	clase externa que se usa en este archivo.
10	import java.io.BufferedReader;	clase externa que se usa en este archivo.
12	import java.io.IOException;	clase externa que se usa en este archivo.
13	import java.io.PrintWriter;	clase externa que se usa en este archivo.
14	import java.io.StringReader;	clase externa que se usa en este archivo.
15	@WebServlet("/auth/login")	URL publica del servlet. Java escuchara esa ruta.
17	public class AuthLoginController extends HttpServlet {	declara la clase principal del archivo.
18	private final UsuarioLoginModel model = new UsuarioLoginModel();	metodo/campo auxiliar de la clase.
22	protected void doOptions(HttpServletRequest request, HttpServletResponse response) {	metodo/campo auxiliar de la clase.
28	protected void doPost(HttpServletRequest request, HttpServletResponse response) throws IOException	responde a peticiones POST, normalmente crear/actualizar.
81	private JsonObject readJson(HttpServletRequest request) throws IOException {	metodo/campo auxiliar de la clase.
90	return Json.createReader(new StringReader(sb.toString())).readObject();	devuelve resultado al metodo que lo llamo.
93	private void writeError(HttpServletResponse response, String mensaje) throws IOException {	metodo/campo auxiliar de la clase.
101	private void setCorsHeaders(HttpServletResponse response) {	metodo/campo auxiliar de la clase.

src/main/java/com/ejemplo/controller/AuthRegisterController.java

Linea	Codigo importante	Que hace
1	package com.ejemplo.controller;	paquete donde vive la clase.
1	import com.ejemplo.model.UsuarioRegisterModel;	clase externa que se usa en este archivo.
3	import jakarta.json.Json;	clase externa que se usa en este archivo.
4	import jakarta.json.JsonObject;	clase externa que se usa en este archivo.

Linea	Codigo importante	Que hace
5	import jakarta.servlet.annotation.WebServlet;	clase externa que se usa en este archivo.
6	import jakarta.servlet.http.HttpServlet;	clase externa que se usa en este archivo.
7	import jakarta.servlet.http.HttpServletRequest;	clase externa que se usa en este archivo.
8	import jakarta.servlet.http.HttpServletResponse;	clase externa que se usa en este archivo.
9	import java.io.BufferedReader;	clase externa que se usa en este archivo.
11	import java.io.IOException;	clase externa que se usa en este archivo.
12	import java.io.PrintWriter;	clase externa que se usa en este archivo.
13	import java.io.StringReader;	clase externa que se usa en este archivo.
14	@WebServlet("/auth/register")	URL publica del servlet. Java escuchara esa ruta.
16	public class AuthRegisterController extends HttpServlet {	declara la clase principal del archivo.
17	private final UsuarioRegisterModel model = new UsuarioRegisterModel();	metodo/campo auxiliar de la clase.
21	protected void doOptions(HttpServletRequest request, HttpServletResponse response) {	metodo/campo auxiliar de la clase.
27	protected void doPost(HttpServletRequest request, HttpServletResponse response) throws IOException	responde a peticiones POST, normalmente crear/actualizar.
62	private JSONObject readJson(HttpServletRequest request) throws IOException {	metodo/campo auxiliar de la clase.
71	return Json.createReader(new StringReader(sb.toString())).readObject();	devuelve resultado al metodo que lo llamo.
74	private void writeJson(HttpServletResponse response, boolean ok, String mensaje) throws IOException	metodo/campo auxiliar de la clase.
88	private boolean isValidEmail(String email) {	metodo/campo auxiliar de la clase.
90	return email != null && email.matches("^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)\$");	devuelve resultado al metodo que lo llamo.
92	private void setCorsHeaders(HttpServletResponse response) {	metodo/campo auxiliar de la clase.

src/main/java/com/ejemplo/controller/AuthRecoverController.java

Linea	Codigo importante	Que hace
1	package com.ejemplo.controller;	paquete donde vive la clase.
1	import com.ejemplo.model.BloqueoUsuarioModel;	clase externa que se usa en este archivo.
3	import com.ejemplo.model.UsuarioRecoveryModel;	clase externa que se usa en este archivo.
4	import jakarta.json.Json;	clase externa que se usa en este archivo.
5	import jakarta.json.JsonObject;	clase externa que se usa en este archivo.
6	import jakarta.json.JsonObjectBuilder;	clase externa que se usa en este archivo.
7	import jakarta.servlet.annotation.WebServlet;	clase externa que se usa en este archivo.
8	import jakarta.servlet.http.HttpServlet;	clase externa que se usa en este archivo.
9	import jakarta.servlet.http.HttpServletRequest;	clase externa que se usa en este archivo.
10	import jakarta.servlet.http.HttpServletResponse;	clase externa que se usa en este archivo.
11	import java.io.BufferedReader;	clase externa que se usa en este archivo.
13	import java.io.IOException;	clase externa que se usa en este archivo.
14	import java.io.PrintWriter;	clase externa que se usa en este archivo.
15	import java.io.StringReader;	clase externa que se usa en este archivo.
16	@WebServlet(urlPatterns = {"/auth/recover/check", "/auth/recover/reset"})	URL publica del servlet. Java escuchara esa ruta.
18	public class AuthRecoverController extends HttpServlet {	declara la clase principal del archivo.

Linea	Codigo importante	Que hace
19	<code>private final UsuarioRecoveryModel model = new UsuarioRecoveryModel();</code>	metodo/campo auxiliar de la clase.
23	<code>protected void doOptions(HttpServletRequest request, HttpServletResponse response) {</code>	metodo/campo auxiliar de la clase.
29	<code>protected void doPost(HttpServletRequest request, HttpServletResponse response) throws IOException</code>	responde a peticiones POST, normalmente crear/actualizar.
73	<code>private void checkAccount(JsonObject body, HttpServletResponse response) throws Exception {</code>	metodo/campo auxiliar de la clase.
112	<code>private void resetPassword(JsonObject body, HttpServletResponse response) throws Exception {</code>	metodo/campo auxiliar de la clase.
149	<code>private JsonObject readJson(HttpServletRequest request) throws IOException {</code>	metodo/campo auxiliar de la clase.
158	<code>return Json.createReader(new StringReader(sb.toString())).readObject();</code>	devuelve resultado al metodo que lo llamo.
161	<code>private void writeJson(HttpServletResponse response, JsonObject json) throws IOException {</code>	metodo/campo auxiliar de la clase.
166	<code>private boolean isValidEmail(String email) {</code>	metodo/campo auxiliar de la clase.
168	<code>return email != null && email.matches("^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)\$");</code>	devuelve resultado al metodo que lo llamo.
170	<code>private void setCorsHeaders(HttpServletResponse response) {</code>	metodo/campo auxiliar de la clase.

src/main/java/com/ejemplo/controller/ContentoController.java

Linea	Codigo importante	Que hace
1	<code>package com.ejemplo.controller;</code>	paquete donde vive la clase.
1	<code>import com.ejemplo.model.ContentoWikiModel;</code>	clase externa que se usa en este archivo.
3	<code>import com.ejemplo.model.ContentoWikiModel.ContentEntry;</code>	clase externa que se usa en este archivo.
4	<code>import com.google.gson.Gson;</code>	clase externa que se usa en este archivo.
5	<code>import jakarta.servlet.annotation.WebServlet;</code>	clase externa que se usa en este archivo.
6	<code>import jakarta.servlet.http.HttpServlet;</code>	clase externa que se usa en este archivo.
7	<code>import jakarta.servlet.http.HttpServletRequest;</code>	clase externa que se usa en este archivo.
8	<code>import jakarta.servlet.http.HttpServletResponse;</code>	clase externa que se usa en este archivo.
9	<code>import java.io.IOException;</code>	clase externa que se usa en este archivo.
11	<code>@WebServlet("/api/content")</code>	URL publica del servlet. Java escuchara esa ruta.
13	<code>public class ContentoController extends HttpServlet {</code>	declara la clase principal del archivo.
14	<code>private final ContentoWikiModel model = new ContentoWikiModel();</code>	metodo/campo auxiliar de la clase.
15	<code>private final Gson gson = new Gson();</code>	metodo/campo auxiliar de la clase.
18	<code>protected void doOptions(HttpServletRequest request, HttpServletResponse response) {</code>	metodo/campo auxiliar de la clase.
24	<code>protected void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException</code>	responde a peticiones GET, normalmente para listar/consultar.
36	<code>protected void doPost(HttpServletRequest request, HttpServletResponse response) throws IOException</code>	responde a peticiones POST, normalmente crear/actualizar.
56	<code>protected void doDelete(HttpServletRequest request, HttpServletResponse response) throws IOException</code>	responde a peticiones DELETE, normalmente borrar.
68	<code>private void setJsonHeaders(HttpServletResponse response) {</code>	metodo/campo auxiliar de la clase.
73	<code>private void setCorsHeaders(HttpServletResponse response) {</code>	metodo/campo auxiliar de la clase.
79	<code>private boolean isBlank(String value) {</code>	metodo/campo auxiliar de la clase.
81	<code>return value == null value.trim().isEmpty();</code>	devuelve resultado al metodo que lo llamo.

Línea	Código importante	Que hace
83	private static class ErrorResponse {	metodo/campo auxiliar de la clase.
85	private final boolean ok;	metodo/campo auxiliar de la clase.
86	private final String error;	metodo/campo auxiliar de la clase.
87	private final String message;	metodo/campo auxiliar de la clase.
88	private ErrorResponse(boolean ok, String error, String message) {	metodo/campo auxiliar de la clase.

src/main/java/com/ejemplo/controller/SolicitudesContenidoController.java

Línea	Código importante	Que hace
1	package com.ejemplo.controller;	paquete donde vive la clase.
1	import com.ejemplo.model.ContenidoWikiModel;	clase externa que se usa en este archivo.
3	import com.ejemplo.model.ContenidoWikiModel.ContenidoRequest;	clase externa que se usa en este archivo.
4	import com.google.gson.Gson;	clase externa que se usa en este archivo.
5	import com.google.gson.JsonObject;	clase externa que se usa en este archivo.
6	import jakarta.servlet.annotation.WebServlet;	clase externa que se usa en este archivo.
7	import jakarta.servlet.http.HttpServlet;	clase externa que se usa en este archivo.
8	import jakarta.servlet.http.HttpServletRequest;	clase externa que se usa en este archivo.
9	import jakarta.servlet.http.HttpServletResponse;	clase externa que se usa en este archivo.
10	import java.io.IOException;	clase externa que se usa en este archivo.
12	@WebServlet("/api/requests")	URL publica del servlet. Java escuchara esa ruta.
14	public class SolicitudesContenidoController extends HttpServlet {	declara la clase principal del archivo.
15	private final ContenidoWikiModel model = new ContenidoWikiModel();	metodo/campo auxiliar de la clase.
16	private final Gson gson = new Gson();	metodo/campo auxiliar de la clase.
19	protected void doOptions(HttpServletRequest request, HttpServletResponse response) {	metodo/campo auxiliar de la clase.
25	protected void doGet(HttpServletRequest request, HttpServletResponse response) throws IOExcepti	responde a peticiones GET, normalmente para listar/consultar.
37	protected void doPost(HttpServletRequest request, HttpServletResponse response) throws IOExcept	responde a peticiones POST, normalmente crear/actualizar.
68	protected void doDelete(HttpServletRequest request, HttpServletResponse response) throws IOExce	responde a peticiones DELETE, normalmente borrar.
80	private void setJsonHeaders(HttpServletResponse response) {	metodo/campo auxiliar de la clase.
85	private void setCorsHeaders(HttpServletResponse response) {	metodo/campo auxiliar de la clase.
91	private boolean isBlank(String value) {	metodo/campo auxiliar de la clase.
93	return value == null value.trim().isEmpty();	devuelve resultado al metodo que lo llamo.
95	private static class ErrorResponse {	metodo/campo auxiliar de la clase.
97	private final boolean ok;	metodo/campo auxiliar de la clase.
98	private final String error;	metodo/campo auxiliar de la clase.
99	private final String message;	metodo/campo auxiliar de la clase.
100	private ErrorResponse(boolean ok, String error, String message) {	metodo/campo auxiliar de la clase.

src/main/java/com/ejemplo/model/ConexionBD.java

Línea	Código importante	Que hace
1	package com.ejemplo.model;	paquete donde vive la clase.

Línea	Código importante	Que hace
1	import java.sql.Connection;	clase externa que se usa en este archivo.
3	import java.sql.DriverManager;	clase externa que se usa en este archivo.
4	import java.sql.SQLException;	clase externa que se usa en este archivo.
5	import java.net.URL;	clase externa que se usa en este archivo.
6	import java.net.URISyntaxException;	clase externa que se usa en este archivo.
7	import java.net.URLDecoder;	clase externa que se usa en este archivo.
8	import java.nio.charset.StandardCharsets;	clase externa que se usa en este archivo.
9	public class ConexionBD {	declara la clase principal del archivo.
11	private static final DatabaseConfig CONFIG = loadConfig();	metodo/campo auxiliar de la clase.
13	private static final String URL = "jdbc:mysql://" + CONFIG.host + ":" + CONFIG.port + "/" + CON	metodo/campo auxiliar de la clase.
23	public static Connection getConnection() throws SQLException {	metodo/campo auxiliar de la clase.
27	return connection;	devuelve resultado al metodo que lo llamo.
29	private static DatabaseConfig loadConfig() {	metodo/campo auxiliar de la clase.
39	return parsed;	devuelve resultado al metodo que lo llamo.
42	return new DatabaseConfig(	devuelve resultado al metodo que lo llamo.
51	private static DatabaseConfig parseDatabaseUrl(String databaseUrl) {	metodo/campo auxiliar de la clase.
66	return new DatabaseConfig(	devuelve resultado al metodo que lo llamo.
74	return null;	devuelve resultado al metodo que lo llamo.
77	private static String firstNonBlank(String... values) {	metodo/campo auxiliar de la clase.
81	return value;	devuelve resultado al metodo que lo llamo.
84	return null;	devuelve resultado al metodo que lo llamo.
86	private static String decode(String value) {	metodo/campo auxiliar de la clase.
88	return URLDecoder.decode(value, StandardCharsets.UTF_8);	devuelve resultado al metodo que lo llamo.
90	private static boolean isBlank(String value) {	metodo/campo auxiliar de la clase.
92	return value == null value.isBlank();	devuelve resultado al metodo que lo llamo.
94	private static class DatabaseConfig {	metodo/campo auxiliar de la clase.
96	private final String host;	metodo/campo auxiliar de la clase.
97	private final String port;	metodo/campo auxiliar de la clase.
98	private final String database;	metodo/campo auxiliar de la clase.
99	private final String user;	metodo/campo auxiliar de la clase.
100	private final String password;	metodo/campo auxiliar de la clase.
101	private DatabaseConfig(String host, String port, String database, String user, String password)	metodo/campo auxiliar de la clase.

src/main/java/com/ejemplo/model/DatabaseSchema.java

Línea	Código importante	Que hace
1	package com.ejemplo.model;	paquete donde vive la clase.
1	import java.sql.Connection;	clase externa que se usa en este archivo.
3	import java.sql.SQLException;	clase externa que se usa en este archivo.
4	import java.sql.SQLSyntaxErrorException;	clase externa que se usa en este archivo.
5	import java.sql.Statement;	clase externa que se usa en este archivo.
6	public class DatabaseSchema {	declara la clase principal del archivo.

Línea	Código importante	Que hace
8	<code>private static volatile boolean initialized = false;</code>	metodo/campo auxiliar de la clase.
10	<code>private DatabaseSchema() {</code>	metodo/campo auxiliar de la clase.
13	<code>public static void ensure(Connection con) throws SQLException {</code>	metodo/campo auxiliar de la clase.
23	<code>try (Statement st = con.createStatement()) {</code>	abre recursos y los cierra automaticamente al terminar.
25	<code>st.executeUpdate("CREATE TABLE IF NOT EXISTS usuarios (" +</code>	ejecuta CREATE/INSERT/UPDATE/DELETE.
35	<code>st.executeUpdate("CREATE TABLE IF NOT EXISTS usuarios_bloqueados (" +</code>	ejecuta CREATE/INSERT/UPDATE/DELETE.
41	<code>st.executeUpdate("CREATE TABLE IF NOT EXISTS contenido_wiki (" +</code>	ejecuta CREATE/INSERT/UPDATE/DELETE.
55	<code>st.executeUpdate("CREATE TABLE IF NOT EXISTS solicitudes_contenido (" +</code>	ejecuta CREATE/INSERT/UPDATE/DELETE.
69	<code>st.executeUpdate("INSERT IGNORE INTO usuarios (username, email, password, rol) VALUES " +</code>	ejecuta CREATE/INSERT/UPDATE/DELETE.
78	<code>private static void addColumnIfMissing(Statement st, String table, String column, String defini</code>	metodo/campo auxiliar de la clase.
81	<code>st.executeUpdate("ALTER TABLE " + table + " ADD COLUMN " + column + " " + definition);</code>	ejecuta CREATE/INSERT/UPDATE/DELETE.

src/main/java/com/ejemplo/model/UsuarioLoginModel.java

Línea	Código importante	Que hace
1	<code>package com.ejemplo.model;</code>	paquete donde vive la clase.
1	<code>import java.sql.Connection;</code>	clase externa que se usa en este archivo.
3	<code>import java.sql.PreparedStatement;</code>	clase externa que se usa en este archivo.
4	<code>import java.sql.ResultSet;</code>	clase externa que se usa en este archivo.
5	<code>public class UsuarioLoginModel {</code>	declara la clase principal del archivo.
7	<code>public int validar(String login, String password) {</code>	metodo/campo auxiliar de la clase.
11	<code>String sql = "SELECT id FROM usuarios WHERE (LOWER(email) = LOWER(?) OR LOWER(username) = LOWER</code>	consulta SQL que se ejecutara contra MySQL.
13	<code>try (Connection con = ConexionBD.getConnection());</code>	abre recursos y los cierra automaticamente al terminar.
15	<code>PreparedStatement ps = con.prepareStatement(sql)) {</code>	consulta preparada; evita concatenar valores directamente.
20	<code>try (ResultSet rs = ps.executeQuery()) {</code>	resultado de una consulta SELECT.
22	<code>return rs.next() ? rs.getInt("id") : -1;</code>	devuelve resultado al metodo que lo llamo.
26	<code>return -1;</code>	devuelve resultado al metodo que lo llamo.
29	<code>public boolean estaBloqueado(String login) {</code>	metodo/campo auxiliar de la clase.
32	<code>return BloqueoUsuarioModel.estaBloqueado(limpiar(login));</code>	devuelve resultado al metodo que lo llamo.
35	<code>return false;</code>	devuelve resultado al metodo que lo llamo.
38	<code>public String obtenerRol(String login) {</code>	metodo/campo auxiliar de la clase.
40	<code>return obtenerCampo(login, "rol", "USER");</code>	devuelve resultado al metodo que lo llamo.
42	<code>public String obtenerUsername(String login) {</code>	metodo/campo auxiliar de la clase.
44	<code>return obtenerCampo(login, "username", "Usuario");</code>	devuelve resultado al metodo que lo llamo.
46	<code>public String obtenerEmail(String login) {</code>	metodo/campo auxiliar de la clase.
48	<code>return obtenerCampo(login, "email", "");</code>	devuelve resultado al metodo que lo llamo.
50	<code>private String obtenerCampo(String login, String campo, String porDefecto) {</code>	metodo/campo auxiliar de la clase.

Línea	Código importante	Que hace
52	String sql = "SELECT " + campo + " FROM usuarios WHERE LOWER(email) = LOWER(?) OR LOWER(usernam	consulta SQL que se ejecutara contra MySQL.
53	try (Connection con = ConexionBD.getConnection());	abre recursos y los cierra automaticamente al terminar.
55	PreparedStatement ps = con.prepareStatement(sql)) {	consulta preparada; evita concatenar valores directamente.
60	try (ResultSet rs = ps.executeQuery()) {	resultado de una consulta SELECT.
62	return rs.next() ? rs.getString(campo) : porDefecto;	devuelve resultado al metodo que lo llamo.
66	return porDefecto;	devuelve resultado al metodo que lo llamo.
69	private String limpiar(String value) {	metodo/campo auxiliar de la clase.
71	return value == null ? "" : value.trim();	devuelve resultado al metodo que lo llamo.

src/main/java/com/ejemplo/model/UsuarioRegisterModel.java

Línea	Código importante	Que hace
1	package com.ejemplo.model;	paquete donde vive la clase.
1	import java.sql.Connection;	clase externa que se usa en este archivo.
3	import java.sql.PreparedStatement;	clase externa que se usa en este archivo.
4	import java.sql.SQLIntegrityConstraintViolationException;	clase externa que se usa en este archivo.
5	public class UsuarioRegisterModel {	declara la clase principal del archivo.
7	public boolean registrar(String username, String email, String password) {	metodo/campo auxiliar de la clase.
14	return false;	devuelve resultado al metodo que lo llamo.
19	return false;	devuelve resultado al metodo que lo llamo.
22	return false;	devuelve resultado al metodo que lo llamo.
24	String sql = "INSERT INTO usuarios (username, email, password, rol) VALUES (?, ?, ?, 'USER')";	consulta SQL que se ejecutara contra MySQL.
26	try (Connection con = ConexionBD.getConnection());	abre recursos y los cierra automaticamente al terminar.
28	PreparedStatement ps = con.prepareStatement(sql)) {	consulta preparada; evita concatenar valores directamente.
33	return ps.executeUpdate() > 0;	ejecuta CREATE/INSERT/UPDATE/DELETE.
35	return false;	devuelve resultado al metodo que lo llamo.
38	return false;	devuelve resultado al metodo que lo llamo.

src/main/java/com/ejemplo/model/UsuarioRecoveryModel.java

Línea	Código importante	Que hace
1	package com.ejemplo.model;	paquete donde vive la clase.
1	import java.sql.Connection;	clase externa que se usa en este archivo.
3	import java.sql.PreparedStatement;	clase externa que se usa en este archivo.
4	import java.sql.ResultSet;	clase externa que se usa en este archivo.
5	public class UsuarioRecoveryModel {	declara la clase principal del archivo.
7	public String obtenerNombrePorEmail(String email) throws Exception {	metodo/campo auxiliar de la clase.
11	return null;	devuelve resultado al metodo que lo llamo.
13	String sql = "SELECT username FROM usuarios WHERE LOWER(email) = LOWER(?) LIMIT 1";	consulta SQL que se ejecutara contra MySQL.

Línea	Código importante	Que hace
15	try (Connection con = ConexionBD.getConnection());	abre recursos y los cierra automáticamente al terminar.
17	PreparedStatement ps = con.prepareStatement(sql) {	consulta preparada; evita concatenar valores directamente.
20	try (ResultSet rs = ps.executeQuery()) {	resultado de una consulta SELECT.
22	return rs.next() ? rs.getString("username") : null;	devuelve resultado al método que lo llamo.
26	public boolean cambiarPassword(String email, String password) throws Exception {	método/campo auxiliar de la clase.
32	return false;	devuelve resultado al método que lo llamo.
36	return false;	devuelve resultado al método que lo llamo.
38	String sql = "UPDATE usuarios SET password = ? WHERE LOWER(email) = LOWER(?)";	consulta SQL que se ejecutara contra MySQL.
40	try (Connection con = ConexionBD.getConnection());	abre recursos y los cierra automáticamente al terminar.
42	PreparedStatement ps = con.prepareStatement(sql) {	consulta preparada; evita concatenar valores directamente.
46	return ps.executeUpdate() > 0;	ejecuta CREATE/INSERT/UPDATE/DELETE.
49	private String normalizarEmail(String email) {	método/campo auxiliar de la clase.
51	return email == null ? "" : email.trim().toLowerCase();	devuelve resultado al método que lo llamo.

src/main/java/com/ejemplo/model/ContenidoWikiModel.java

Línea	Código importante	Que hace
1	package com.ejemplo.model;	paquete donde vive la clase.
1	import java.sql.Connection;	clase externa que se usa en este archivo.
3	import java.sql.PreparedStatement;	clase externa que se usa en este archivo.
4	import java.sql.ResultSet;	clase externa que se usa en este archivo.
5	import java.sql.Statement;	clase externa que se usa en este archivo.
6	import java.sql.Timestamp;	clase externa que se usa en este archivo.
7	import java.time.Instant;	clase externa que se usa en este archivo.
8	import java.util.ArrayList;	clase externa que se usa en este archivo.
9	import java.util.List;	clase externa que se usa en este archivo.
10	public class ContenidoWikiModel {	declara la clase principal del archivo.
12	public List<ContenidoEntry> listarContenido(String pageKey) throws Exception {	abre recursos y los cierra automáticamente al terminar.
14	String sql = "SELECT id, page_key, titulo, cuerpo, autor_nombre, autor_email, autor_rol, creado	consulta SQL que se ejecutara contra MySQL.
18	try (Connection con = ConexionBD.getConnection());	abre recursos y los cierra automáticamente al terminar.
20	PreparedStatement ps = con.prepareStatement(sql) {	consulta preparada; evita concatenar valores directamente.
24	try (ResultSet rs = ps.executeQuery()) {	resultado de una consulta SELECT.
30	return entries;	devuelve resultado al método que lo llamo.
34	public ContenidoEntry guardarContenido(ContenidoEntry entry) throws Exception {	abre recursos y los cierra automáticamente al terminar.
40	String sql = "UPDATE contenido_wiki SET page_key = ?, titulo = ?, cuerpo = ?, autor_nombre = ?,	consulta SQL que se ejecutara contra MySQL.
41	try (Connection con = ConexionBD.getConnection());	abre recursos y los cierra automáticamente al terminar.

Línea	Código importante	Que hace
42	PreparedStatement ps = con.prepareStatement(sql)) {	consulta preparada; evita concatenar valores directamente.
52	return buscarContenido(id);	devuelve resultado al metodo que lo llamo.
54	String sql = "INSERT INTO contenido_wiki (page_key, titulo, cuerpo, autor_nombre, autor_email,	consulta SQL que se ejecutara contra MySQL.
56	try (Connection con = ConexionBD.getConnection());	abre recursos y los cierra automaticamente al terminar.
57	PreparedStatement ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {	consulta preparada; evita concatenar valores directamente.
65	try (ResultSet keys = ps.getGeneratedKeys()) {	resultado de una consulta SELECT.
68	return buscarContenido(keys.getInt(1));	devuelve resultado al metodo que lo llamo.
72	return entry;	abre recursos y los cierra automaticamente al terminar.
75	public boolean borrarContenido(int id) throws Exception {	metodo/campo auxiliar de la clase.
77	try (Connection con = ConexionBD.getConnection());	abre recursos y los cierra automaticamente al terminar.
78	PreparedStatement ps = con.prepareStatement("DELETE FROM contenido_wiki WHERE id = ?") {	consulta preparada; evita concatenar valores directamente.
80	return ps.executeUpdate() > 0;	ejecuta CREATE/INSERT/UPDATE/DELETE.
83	public List<ContenidoRequest> listarSolicitudes(String pageKey) throws Exception {	metodo/campo auxiliar de la clase.
85	String sql = "SELECT id, page_key, titulo, mensaje, remitente_nombre, remitente_email, remitent	consulta SQL que se ejecutara contra MySQL.
89	try (Connection con = ConexionBD.getConnection());	abre recursos y los cierra automaticamente al terminar.
91	PreparedStatement ps = con.prepareStatement(sql)) {	consulta preparada; evita concatenar valores directamente.
95	try (ResultSet rs = ps.executeQuery()) {	resultado de una consulta SELECT.
101	return requests;	devuelve resultado al metodo que lo llamo.
105	public ContenidoRequest crearSolicitud(ContenidoRequest request) throws Exception {	metodo/campo auxiliar de la clase.
107	String sql = "INSERT INTO solicitudes_contenido (page_key, titulo, mensaje, remitente_nombre, r	consulta SQL que se ejecutara contra MySQL.
108	try (Connection con = ConexionBD.getConnection());	abre recursos y los cierra automaticamente al terminar.
109	PreparedStatement ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {	consulta preparada; evita concatenar valores directamente.
118	try (ResultSet keys = ps.getGeneratedKeys()) {	resultado de una consulta SELECT.
121	return buscarSolicitud(keys.getInt(1));	devuelve resultado al metodo que lo llamo.
125	return request;	devuelve resultado al metodo que lo llamo.
127	public boolean actualizarEstadoSolicitud(int id, String status) throws Exception {	metodo/campo auxiliar de la clase.
129	try (Connection con = ConexionBD.getConnection());	abre recursos y los cierra automaticamente al terminar.
130	PreparedStatement ps = con.prepareStatement("UPDATE solicitudes_contenido SET estado = ? WHERE	consulta preparada; evita concatenar valores directamente.
133	return ps.executeUpdate() > 0;	ejecuta CREATE/INSERT/UPDATE/DELETE.
136	public boolean borrarSolicitud(int id) throws Exception {	metodo/campo auxiliar de la clase.
138	try (Connection con = ConexionBD.getConnection());	abre recursos y los cierra automaticamente al terminar.

Línea	Código importante	Que hace
139	PreparedStatement ps = con.prepareStatement("DELETE FROM solicitudes_contenido WHERE id = ?");	consulta preparada; evita concatenar valores directamente.
141	return ps.executeUpdate() > 0;	ejecuta CREATE/INSERT/UPDATE/DELETE.
144	private ContenidoEntry buscarContenido(int id) throws Exception {	abre recursos y los cierra automáticamente al terminar.
146	try (Connection con = ConexionBD.getConnection());	abre recursos y los cierra automáticamente al terminar.
147	PreparedStatement ps = con.prepareStatement("SELECT id, page_key, titulo, cuerpo, autor_nombre,	consulta preparada; evita concatenar valores directamente.
149	try (ResultSet rs = ps.executeQuery()) {	resultado de una consulta SELECT.
150	return rs.next() ? mapContenido(rs) : null;	devuelve resultado al método que lo llamo.
154	private ContenidoRequest buscarSolicitud(int id) throws Exception {	método/campo auxiliar de la clase.
156	try (Connection con = ConexionBD.getConnection());	abre recursos y los cierra automáticamente al terminar.

Este archivo tiene 115 líneas importantes detectadas. Las restantes son setters/getters, cierres de bloques o conversiones repetidas.

src/main/java/com/ejemplo/filter/NoCacheFilter.java

Línea	Código importante	Que hace
1	package com.ejemplo.filter;	paquete donde vive la clase.
1	import jakarta.servlet.Filter;	clase externa que se usa en este archivo.
3	import jakarta.servlet.FilterChain;	clase externa que se usa en este archivo.
4	import jakarta.servlet.ServletException;	clase externa que se usa en este archivo.
5	import jakarta.servlet.ServletRequest;	clase externa que se usa en este archivo.
6	import jakarta.servlet.ServletResponse;	clase externa que se usa en este archivo.
7	import jakarta.servlet.annotation.WebFilter;	clase externa que se usa en este archivo.
8	import jakarta.servlet.http.HttpServletRequest;	clase externa que se usa en este archivo.
9	import java.io.IOException;	clase externa que se usa en este archivo.
13	public class NoCacheFilter implements Filter {	declara la clase principal del archivo.
14	private static final String DISABLE_CACHE_FLAG = "true";	método/campo auxiliar de la clase.
18	public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)	método/campo auxiliar de la clase.
29	private boolean shouldDisableCache() {	método/campo auxiliar de la clase.
32	return DISABLE_CACHE_FLAG.equalsIgnoreCase(envValue);	devuelve resultado al método que lo llamo.

9. Base de datos

La base de datos es MySQL. DatabaseSchema.java puede crear tablas automáticamente al iniciar si no existen. El SQL inicial también está en mysql/init/01-bd1.sql.

Tabla	Para que sirve	Campos importantes
usuarios	Cuentas de la web.	id, username, email, password, rol, bloqueado, creado_en
usuarios_bloqueados	Correos bloqueados por admin.	email, motivo, creado_en
contenido_wiki	Contenido publicado por admin/editor.	page_key, titulo, cuerpo, autor_nombre, autor_email, autor_rol
solicitudes_contenido	Mensajes enviados por usuarios.	page_key, titulo, mensaje, remitente_nombre, remitente_email, estado

10. Flujos para explicar en un examen

Login

- ? El usuario escribe email/usuario y contraseña en login.html.
- ? login.js escucha el submit, evita que el formulario recargue la pagina y hace fetch a /auth/login.
- ? AuthLoginController recibe JSON, llama a UsuarioLoginModel y comprueba en MySQL.
- ? Si es correcto, Java devuelve datos del usuario y JS los guarda en localStorage.
- ? Despues se redirige segun rol: usuario normal a dashboard, admin/editor a admin o pagina correspondiente.

Solicitudes de usuario

- ? En una pagina interna, script.js detecta data-page para saber en que seccion estas.
- ? El usuario escribe titulo y mensaje. JS crea una solicitud con pageKey y datos del usuario.
- ? Se envia por POST a /api/requests.
- ? SolicitudesContenidoController llama a ContenidoWikiModel.crearSolicitud y guarda en solicitudes_contenido.
- ? El panel admin lee /api/requests y muestra esas solicitudes.

Contenido publicado por admin

- ? El admin escribe titulo y cuerpo en admin.html.
- ? admin.js envia POST a /api/content.
- ? ContenidoController llama a ContenidoWikiModel.guardarContenido.
- ? Se guarda en contenido_wiki con page_key.
- ? Cuando el usuario abre esa pagina, script.js hace GET /api/content y filtra por page_key para mostrarlo debajo del bloque correspondiente.

11. Chuleta rapida

Pregunta	Respuesta corta
Donde esta la conexion a MySQL?	ConexionBD.java.
Quien crea tablas si faltan?	DatabaseSchema.java.
Quien guarda contenido de comunidad?	ContenidoWikiModel.java llamado desde ContenidoController.
Quien guarda solicitudes?	ContenidoWikiModel.java llamado desde SolicitudesContenidoController.
Donde esta el panel admin visual?	admin.html + admin.css + admin.js.
Como sabe JS en que pagina esta?	Por el atributo data-page del body o la pagina actual.
Donde se guarda la sesion en navegador?	localStorage: anime-authenticated, anime-username, anime-user-email, anime-user-role.
Que hace NoCacheFilter?	Si APP_DISABLE_CACHE=true, fuerza al navegador a no cachear respuestas.

12. Conteo de archivos principales

Archivo	Lineas	Funcion
src/main/webapp/index.html	98	Archivo clave del proyecto
src/main/webapp/dashboard.html	124	Archivo clave del proyecto
src/main/webapp/admin.html	116	Archivo clave del proyecto
src/main/webapp/login.html	43	Archivo clave del proyecto

Archivo	Lineas	Funcion
src/main/webapp/register.html	53	Archivo clave del proyecto
src/main/webapp/forgot-password.html	54	Archivo clave del proyecto
src/main/webapp/Assets/css/dashboard.css	1763	Archivo clave del proyecto
src/main/webapp/Assets/css/admin.css	440	Archivo clave del proyecto
src/main/webapp/Assets/css/login.css	237	Archivo clave del proyecto
src/main/webapp/Assets/js/script.js	1080	Archivo clave del proyecto
src/main/webapp/Assets/js/admin.js	633	Archivo clave del proyecto
src/main/webapp/Assets/js/auth-helper.js	245	Archivo clave del proyecto
src/main/java/com/ejemplo/model/ContenidoWikiModel.java	286	Archivo clave del proyecto
src/main/java/com/ejemplo/model/ConexionBD.java	111	Archivo clave del proyecto
src/main/java/com/ejemplo/model/DatabaseSchema.java	89	Archivo clave del proyecto

13. Fragmentos de codigo con numeros de linea

Estos fragmentos sirven para ubicarte rapido cuando estudies. En el proyecto real tienes el archivo completo.

src/main/webapp/Assets/js/login.js

```

0001: document.addEventListener("DOMContentLoaded", () => {
0002:   injectSiteFooter();
0003:
0004:   const form = document.getElementById("loginForm");
0005:   const error = document.getElementById("error");
0006:
0007:   form?.addEventListener("submit", async (event) => {
0008:     event.preventDefault();
0009:
0010:     const login = document.getElementById("login").value.trim();
0011:     const password = document.getElementById("password").value.trim();
0012:
0013:     if (!login || !password) {
0014:       error.textContent = "Completa usuario y contrasena.";
0015:       return;
0016:     }
0017:
0018:     const remoteResult = await tryRemoteLogin(login, password);
0019:
0020:     if (remoteResult.ok) {
0021:       const remoteUser = {
0022:         username: remoteResult.username || login,
0023:         email: remoteResult.email || login,
0024:         password,
0025:         role: remoteResult.role || "user"
0026:       };
0027:
0028:       if (window.AuthHelper) {
0029:         window.AuthHelper.upsertUser(remoteUser);
0030:         window.AuthHelper.saveSession(remoteUser);
0031:       }
0032:
0033:       redirectAfterLogin();
0034:       return;
0035:     }
0036:
0037:     const localResult = window.AuthHelper
0038:       ? window.AuthHelper.verifyLocalUser(login, password)
0039:       : { ok: false };
0040:
0041:   });
0042:
0043:   function getApiBase() {
0044:     return window.location.protocol === "file:"
0045:       ? "http://localhost:8080"
0046:       : window.location.origin;
0047:   }
0048:
0049:   async function tryRemoteLogin(login, password) {
0050:     try {
0051:       const res = await fetch(`${getApiBase()}/auth/login`, {
0052:         method: "POST",
0053:         headers: {
0054:           "Content-Type": "application/json"
0055:         }
0056:       });
0057:
0058:       return safeJson(res);
0059:     } catch {
0060:       return { ok: false };
0061:     }
0062:   }
0063:
0064:   async function safeJson(response) {
0065:     try {
0066:       return await response.json();
0067:     } catch {
0068:       return null;
0069:     }
0070:   }

```

```

0093:   } catch (error) {
0096: }
0097:
0098: function redirectAfterLogin() {
0099:   window.location.href = "dashboard.html";
0100: }
0101:
0102: function injectSiteFooter() {
0103:   if (document.querySelector(".site-footer")) {
0104:     return;
0105:   }

```

src/main/webapp/Assets/js/script.js

```

0001: const AUTH_STORAGE_KEY = "anime-wiki-authenticated";
0002: const USERNAME_STORAGE_KEY = "anime-wiki-username";
0003: const USER_EMAIL_STORAGE_KEY = "anime-wiki-user-email";
0004: const USER_ROLE_STORAGE_KEY = "anime-wiki-user-role";
0005: const CONTENT_STORAGE_KEY = "anime-page-content-v1";
0006: const CONTENT_REQUESTS_STORAGE_KEY = "anime-page-requests-v1";
0007: const SITE_FAVICON_PATH = "Assets/img/logo-circle.png?v=1";
0008: const PAGE_AUDIO_CONFIGS = [
0009:   {
0010:     themeClass: "theme-one-piece",
0011:     audioPath: "Assets/one-piece-theme.mp3?v=1",
0012:     storageKey: "one-piece-audio-state-v1",
0013:     audioId: "onePieceThemeAudio",
0014:     buttonId: "onePieceAudioToggle",
0015:     seriesName: "One Piece"
0016:   },
0017:   {
0018:     themeClass: "theme-naruto",
0019:     audioPath: "Assets/naruto-theme.mp3?v=1",
0020:     storageKey: "naruto-audio-state-v1",
0021:     audioId: "narutoThemeAudio",
0022:     buttonId: "narutoAudioToggle",
0023:     seriesName: "Naruto"
0024:   },
0025:   {
0026:     themeClass: "theme-bleach",
0027:     audioPath: "Assets/bleach-theme.mp3?v=1",
0028:     storageKey: "bleach-audio-state-v1",
0029:     audioId: "bleachThemeAudio",
0030:     buttonId: "bleachAudioToggle",
0031:     seriesName: "Bleach"
0032:   }
0033: ];
0034: const SERIES_ASIDE_VIDEO_CONFIGS = [
0035:   {
0036:     themeClass: "theme-one-piece",
0037:     boxId: "onePieceVideoBox",
0038:     heading: "Videos de One Piece",
0039:     videos: [
0040:       {
0092:     };
0093:
0094: function redirectToLogin() {
0095:   window.location.href = "login.html";
0096: }
0097:
0098: function isAuthenticated() {
0099:   return localStorage.getItem(AUTH_STORAGE_KEY) === "true";
0100: }
0101:
0102: function getCurrentRole() {
0103:   return String(localStorage.getItem(USER_ROLE_STORAGE_KEY) || "user").toLowerCase();
0104: }
0105:
0106: function getCurrentUserName() {
0107:   return localStorage.getItem(USERNAME_STORAGE_KEY) || "Invitado";
0108: }
0109:
0110: function getCurrentUserEmail() {
0111:   return localStorage.getItem(USER_EMAIL_STORAGE_KEY) || "";
0112: }
0113:
0114: function getApiBase() {
0115:   return window.location.protocol === "file:"
0116:     ? "http://localhost:8080"
0117:     : window.location.origin;
0118: }
0119:
0120: async function safeJson(response) {
0121:   try {
0122:     return await response.json();
0123:   } catch (error) {
0126: }
0127:
0128: async function syncCommunityStateFromServer() {
0129:   try {
0130:     const [contentResponse, requestsResponse] = await Promise.all([
0131:       fetch(`${getApiBase()}/api/content`),
0132:       fetch(`${getApiBase()}/api/requests`)
0133:     ]);
0134:
0135:     if (contentResponse.ok) {
0151:   }
0152:

```

```

0153: async function persistContentEntry(entry) {
0154:   try {
0155:     const response = await fetch(`${getApiBase()}/api/content`, {
0156:       method: "POST",
0157:       headers: { "Content-Type": "application/json" },
0158:       body: JSON.stringify(entry)
0159:     })
0160:   } catch (error) {
0161:     // El localStorage queda como respaldo si el servidor no responde.
0162:   }
0163:   return entry;
0164: }

```

src/main/webapp/Assets/js/admin.js

```

0001: const AUTH_STORAGE_KEY = "anime-wiki-authenticated";
0002: const USERNAME_STORAGE_KEY = "anime-wiki-username";
0003: const USER_ROLE_STORAGE_KEY = "anime-wiki-user-role";
0004: const USER_EMAIL_STORAGE_KEY = "anime-wiki-user-email";
0005: const CONTENT_STORAGE_KEY = "anime-page-content-v1";
0006: const CONTENT_REQUESTS_STORAGE_KEY = "anime-page-requests-v1";
0007:
0008: const PAGE_OPTIONS = [
0009:   { key: "one-piece-arcos", label: "One Piece - Arcos" },
0010:   { key: "one-piece-frutas-del-diablo", label: "One Piece - Frutas del diablo" },
0011:   { key: "one-piece-mares", label: "One Piece - Mares" },
0012:   { key: "one-piece-sichibukais", label: "One Piece - Sichibukais" },
0013:   { key: "one-piece-tripulacion", label: "One Piece - Tripulacion" },
0014:   { key: "one-piece-yonkos", label: "One Piece - Yonkos" },
0015:   { key: "naruto-equipo-7", label: "Naruto - Equipo 7" },
0016:   { key: "naruto-ojos", label: "Naruto - Ojos" },
0017:   { key: "naruto-akatsuki", label: "Naruto - Akatsuki" },
0018:   { key: "naruto-bijus", label: "Naruto - Bijus" },
0019:   { key: "naruto-hokages", label: "Naruto - Hokages" },
0020:   { key: "naruto-clanes", label: "Naruto - Clanes" },
0021:   { key: "bleach-shinigamis", label: "Bleach - Shinigamis" },
0022:   { key: "bleach-hollows", label: "Bleach - Hollows" },
0023:   { key: "bleach-vizards", label: "Bleach - Vizards" },
0024:   { key: "bleach-quincys", label: "Bleach - Quincys" },
0025:   { key: "bleach-zanpakutos", label: "Bleach - Zanpakutos" },
0026:   { key: "bleach-bankais", label: "Bleach - Bankais" }
0027: ];
0028:
0029: function isAuthenticated() {
0030:   return localStorage.getItem(AUTH_STORAGE_KEY) === "true";
0031: }
0032:
0033: function getCurrentRole() {
0034:   return String(localStorage.getItem(USER_ROLE_STORAGE_KEY) || "user").toLowerCase();
0035: }
0036:
0037: function getCurrentUserName() {
0038:   return localStorage.getItem(USERNAME_STORAGE_KEY) || "Usuario";
0039: }
0040:
0041: function getCurrentUserEmail() {
0042:   return localStorage.getItem(USER_EMAIL_STORAGE_KEY) || "";
0043: }
0044:
0045: function getApiBase() {
0046:   return window.location.protocol === "file:"
0047:     ? "http://localhost:8080"
0048:     : window.location.origin;
0049: }
0050:
0051: async function safeJson(response) {
0052:   try {
0053:     return await response.json();
0054:   } catch (error) {
0055:   }
0056: }
0057:
0058: async function syncAdminStateFromServer() {
0059:   try {
0060:     const [contentResponse, requestsResponse] = await Promise.all([
0061:       fetch(`${getApiBase()}/api/content`),
0062:       fetch(`${getApiBase()}/api/requests`)
0063:     ]);
0064:   }
0065:   if (contentResponse.ok) {
0066:   }
0067: }
0068:
0069: async function persistEntry(entry) {
0070:   try {
0071:     const response = await fetch(`${getApiBase()}/api/content`, {
0072:       method: "POST",
0073:       headers: { "Content-Type": "application/json" },
0074:       body: JSON.stringify(entry)
0075:     })
0076:   }
0077: }
0078:
0079: async function removeEntryRemote(entryId) {
0080:   try {
0081:     await fetch(`${getApiBase()}/api/content?id=${encodeURIComponent(entryId)}`, {
0082:       method: "DELETE"
0083:     });
0084:   } catch (error) {
0085:   }
0086: }
0087:
0088: async function persistRequestStatus(requestId, status) {
0089:   try {

```

```

0113:     await fetch(`${getApiBase()}/api/requests`, {
0114:         method: "POST",
0115:         headers: { "Content-Type": "application/json" },
0116:         body: JSON.stringify({ action: "status", id: requestId, status })
0121:     }
0122:
0123: async function removeRequestRemote(requestId) {
0124:     try {
0125:         await fetch(`${getApiBase()}/api/requests?id=${encodeURIComponent(requestId)}`, {
0126:             method: "DELETE"
0127:         });

```

src/main/java/com/ejemplo/controller/ContenidoController.java

```

0001: package com.ejemplo.controller;
0002:
0003: import com.ejemplo.model.ContenidoWikiModel;
0004: import com.ejemplo.model.ContenidoWikiModel.ContenidoEntry;
0005: import com.google.gson.Gson;
0006: import jakarta.servlet.annotation.WebServlet;
0007: import jakarta.servlet.http.HttpServlet;
0008: import jakarta.servlet.http.HttpServletRequest;
0009: import jakarta.servlet.http.HttpServletResponse;
0010:
0011: import java.io.IOException;
0012:
0013: @WebServlet("/api/content")
0014: public class ContenidoController extends HttpServlet {
0015:     private final ContenidoWikiModel model = new ContenidoWikiModel();
0016:     private final Gson gson = new Gson();
0017:
0018:     @Override
0019:     protected void doOptions(HttpServletRequest request, HttpServletResponse response) {
0020:         setCorsHeaders(response);
0021:         response.setStatus(HttpServletResponse.SC_NO_CONTENT);
0022:     }
0023:
0024:     @Override
0025:     protected void doGet(HttpServletRequest request, HttpServletResponse response)
0026:     throws IOException {
0027:         setJsonHeaders(response);
0028:         try {
0029:             response.getWriter().print(gson.toJson(model.listarContenido(request.
0030: getParameter("pageKey"))));
0031:         } catch (Exception e) {
0032:             e.printStackTrace();
0033:             response.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
0034:             response.getWriter().print(gson.toJson(new ErrorResponse(false, e.getClass().
0035: getSimpleName(), e.getMessage())));
0036:         }
0037:     }
0038:
0039:     @Override
0040:     protected void doPost(HttpServletRequest request, HttpServletResponse response)
0041:     throws IOException {
0042:         setJsonHeaders(response);
0043:         try {
0044:             ContenidoEntry entry = gson.fromJson(request.getReader(), ContenidoEntry.
0045: class);
0046:
0047:             @Override
0048:             protected void doDelete(HttpServletRequest request, HttpServletResponse response)
0049:             throws IOException {
0050:                 setJsonHeaders(response);
0051:                 try {
0052:                     int id = Integer.parseInt(request.getParameter("id"));

```

src/main/java/com/ejemplo/model/ContenidoWikiModel.java

```

0001: package com.ejemplo.model;
0002:
0003: import java.sql.Connection;
0004: import java.sql.PreparedStatement;
0005: import java.sql.ResultSet;
0006: import java.sql.Statement;
0007: import java.sql.Timestamp;
0008: import java.time.Instant;
0009: import java.util.ArrayList;
0010: import java.util.List;
0011:
0012: public class ContenidoWikiModel {
0013:
0014:     public List<ContenidoEntry> listarContenido(String pageKey) throws Exception {
0015:         String sql = "SELECT id, page_key, titulo, cuerpo, autor_nombre, autor_email,
0016: autor_rol, creado_en, actualizado_en " +
0017: "FROM contenido_wiki " +
0018: "(isBlank(pageKey) ? "" : "WHERE page_key = ? ") +
0019: "ORDER BY actualizado_en DESC, creado_en DESC";
0020:
0021:         try (Connection con = ConexionBD.getConnection();
0022:             PreparedStatement ps = con.prepareStatement(sql)) {
0023:             if (!isBlank(pageKey)) {
0024:                 ps.setString(1, pageKey.trim());
0025:             }
0026:
0027:             try (ResultSet rs = ps.executeQuery()) {

```



```

0027:         List<ContenidoEntry> entries = new ArrayList<>();
0028:         while (rs.next()) {
0029:             entries.add(mapContenido(rs));
0030:         }
0031:         return entries;
0032:     }
0033: }
0034: }
0035:
0036: public ContenidoEntry guardarContenido(ContenidoEntry entry) throws Exception {
0037:     Integer id = parseId(entry.id);
0038:     String autorRol = normalizeRole(entry.authorRole);
0039:
0040:     if (id != null && existeContenido(id)) {
0041:         String sql = "UPDATE contenido_wiki SET page_key = ?, titulo = ?, cuerpo = ?,
0042:             autor_nombre = ?, autor_email = ?, autor_rol = ? WHERE id = ?";
0043:         try (Connection con = ConexionBD.getConnection();
0044:             PreparedStatement ps = con.prepareStatement(sql)) {
0045:             ps.setString(1, clean(entry.pageKey));
0046:         }
0047:
0048:         String sql = "INSERT INTO contenido_wiki (page_key, titulo, cuerpo, autor_nombre,
0049:             autor_email, autor_rol) VALUES (?, ?, ?, ?, ?, ?)";
0050:         try (Connection con = ConexionBD.getConnection();
0051:             PreparedStatement ps = con.prepareStatement(sql, Statement.
0052:                 RETURN_GENERATED_KEYS)) {
0053:             ps.setString(1, clean(entry.pageKey));
0054:         }
0055:
0056:         public List<ContenidoRequest> listarSolicitudes(String pageKey) throws Exception {
0057:             String sql = "SELECT id, page_key, titulo, mensaje, remitente_nombre,
0058:                 remitente_email, remitente_rol, estado, creado_en " +
0059:                 "FROM solicitudes_contenido " +
0060:                 "(isBlank(pageKey) ? "" : "WHERE page_key = ? ") +
0061:                 "ORDER BY FIELD(estado, 'PENDIENTE', 'REVISADA'), creado_en DESC";
0062:
0063:             public ContenidoRequest crearSolicitud(ContenidoRequest request) throws Exception {
0064:                 String sql = "INSERT INTO solicitudes_contenido (page_key, titulo, mensaje,
0065:                     remitente_nombre, remitente_email, remitente_rol, estado) VALUES (?, ?, ?, ?, ?, ?, ?)";
0066:                 try (Connection con = ConexionBD.getConnection();
0067:                     PreparedStatement ps = con.prepareStatement(sql, Statement.
0068:                         RETURN_GENERATED_KEYS)) {
0069:                     ps.setString(1, clean(request.pageKey));

```